VECTOR

MICROSAR CRYPTO

Technical Reference

CRYPTO VHSM

Version 2.0.0

Authors	Tobias Finke
Status	Released

Document Information

History

Author	Date	Version	Remarks
Finke, Tobias	2017-12-15	1.00.00	Initial creation of Technical Reference

Finke, Tobias	2017-02-15	1.01.00	Driver is now Hardware independent
Finke, Tobias	2017-05-07	1.02.00	Release of component
Ritter, Philipp	2018-09-24	2.00.00	Chapter 3.7.1, 3.7.4, 3.8 and 5.3 added.
Peukert, Frederik	2018-09-25	2.00.00	Description of synchronization added

Reference Documents

No.	Source	Title	Version
[1]	AUTOSAR	AUTOSAR_CryptoDriver.pdf	4.3.0
[2]	AUTOSAR	AUTOSAR_SWS_DET.pdf	4.3.0
[3]	AUTOSAR	AUTOSAR_TR_BSWModuleList.pdf	4.3.0
[4]	HIS	2009-04-01 SHE Functional Specification v1.1 (rev439)	1.1

Contents

1 Component History 7

2 Introduction 8
2.1 Architecture Overview 8

3 Functional Description 10
3.1 Features 10
3.1.1 Deviations 10
3.1.2 Additions/ Extensions 10
3.1.3 Limitations 11
3.1.3.1 Cryptographic Algorithms and Modes 11
3.1.3.2 Certificate Handling 11
3.1.3.3 Key Generation 11
3.1.3.4 AEAD Det Checks 11
3.1.3.5 32 Bit Platform Support Only 11
3.2 Synchronization with vHsm 11
3.3 Initialization 11
3.4 Main Functions 12
3.5 Error Handling 12
3.5.1 Development Error Reporting (DET) 12
3.5.2 Algorithm Parameter Overview 14
3.6 Key Usage 15
3.6.1 Custom Key Elements 15
3.6.2 Key Element Values 16
3.7 vHsm 17
3.7.1 Pre Config File 17
3.7.2 Key Mapping 17
3.7.3 Crypto Driver Objects 18
3.7.4 IPC Versions 18
3.7.5 Memory Access 18
3.7.6 Special services 19
3.7.6.1 Version Info 19
3.7.6.2 UID 19
3.7.6.3 Error Log 19
3.7.6.4 Secure Boot 19
3.7.6.5 Secure Software Download 20
3.7.6.6 Hardware Dependent Features 21
3.7.7 Limitations 21
3.7.7.1 Secure Boot 21

3.7.7.2 Write-Once Keys 21
3.8 Wait Loop Callout 21

4 Integration 22
4.1 Scope of Delivery 22
4.1.1 Static Files 22
4.1.2 Dynamic Files 22
4.2 Critical Sections 22

5 API Description 24
5.1 Services provided by CRYPTO 24
5.1.1 Crypto_30_vHsm_Init 24
5.1.2 Crypto_30_vHsm_InitMemory 24
5.1.3 Crypto_30_vHsm_GetVersionInfo 25
5.1.4 Crypto_30_vHsm_ProcessJob 25
5.1.5 Crypto_30_vHsm_CancelJob 26
5.1.6 Crypto_30_WaitForHsmRam 27
5.2 Key Management Functions 27
5.2.1 Crypto_30_vHsm_KeyCopy 27
5.2.2 Crypto_30_vHsm_KeyElementCopy 28
5.2.3 Crypto_30_vHsm_KeyElementIdsGet 29
5.2.4 Crypto_30_vHsm_KeyElementSet 30
5.2.5 Crypto_30_vHsm_KeyValidSet 30
5.2.6 Crypto_30_vHsm_KeyElementGet 31
5.2.7 Crypto_30_vHsm_RandomSeed 32
5.2.8 Crypto_30_vHsm_KeyGenerate 32
5.2.9 Crypto_30_vHsm_KeyDerive 33
5.2.10 Crypto_30_vHsm_KeyExchangeCalcPubVal 34
5.2.11 Crypto_30_vHsm_KeyExchangeCalcSecret 34
5.2.12 Crypto_30_vHsm_CertificateParse 35
5.2.13 Crypto_30_vHsm_CertificateVerify 36
5.3 Callout Functions 36
5.3.1 Wait Loop Callout 37
5.4 Services needed by CRYPTO 37
5.5 Services used by CRYPTO 37

6 Configuration 38
6.1 Configuration Variants 38

7 Glossary and Abbreviations 39
7.1 Glossary 39

7.2 Abbreviations 39

8 Contact 40

Illustrations

Figure 2-1 AUTOSAR 4.2 Architecture Overview 8
Figure 2-2 Interfaces to adjacent modules of the CRYPTO 9

Tables

Table 1-1 Component history 7
Table 3-1 Supported AUTOSAR standard conform features 10
Table 3-2 Not supported AUTOSAR standard conform features 10
Table 3-3 Features provided beyond the AUTOSAR standard 11
Table 3-4 Service IDs 13
Table 3-5 Errors reported to DET 13
Table 3-6 Overview of the required algorithm parameter 14
Table 3-7 Custom Key Elements 16
Table 3-8 Key Element Values 17
Table 3-9 Recommended settings for secure boot key elements 20
Table 4-1 Static files 22
Table 4-2 Generated files 22
Table 5-1 Crypto_30_vHsm_Init 24
Table 5-2 Crypto_30_vHsm_InitMemory 25
Table 5-3 Crypto_30_vHsm_GetVersionInfo 25
Table 5-4 Crypto_30_vHsm_ProcessJob 26
Table 5-5 Crypto_30_vHsm_CancelJob 26
Table 5-6 Crypto_30_vHsm_WaitForHsmRam 27
Table 5-7 Crypto_30_vHsm_KeyCopy 28
Table 5-8 Crypto_30_vHsm_KeyElementCopy 29
Table 5-9 Crypto_30_vHsm_KeyElementIdsGet 30
Table 5-10 Crypto_30_vHsm_KeyElementSet 30
Table 5-11 Crypto_30_vHsm_KeyValidSet 31
Table 5-12 Crypto_30_vHsm_KeyElementGet 32
Table 5-13 Crypto_30_vHsm_RandomSeed 32
Table 5-14 Crypto_30_vHsm_KeyGenerate 33
Table 5-15 Crypto_30_vHsm_KeyDerive 34
Table 5-16 Crypto_30_vHsm_KeyExchangeCalcPubVal 34
Table 5-17 Crypto_30_vHsm_KeyExchangeCalcSecret 35
Table 5-18 Crypto_30_vHsm_CertificateParse 36
Table 5-19 Crypto_30_vHsm_CertificateVerify 36
Table 5-20 Key Verification Callout 37
Table 5-21 Services used by the CRYPTO 37
Table 7-1 Glossary 39
Table 7-2 Abbreviations 39

1 Component History

The component history gives an overview over the important milestones that are supported in the different versions of the component.

Component Version	New Features
1.00	Initial beta release
1.01	Feature completion
1.02	Release of component
2.00	IPC Update, Wait Loop Callout

Table 1-1 Component history

2 Introduction

This document describes the functionality, API and configuration of the AUTOSAR BSW module CRYPTO as specified in [1].

Supported AUTOSAR Release*:	4.3	
Supported Configuration Variants:	pre-compile	
Vendor ID:	CRYPTO_VENDOR_ID	30 decimal (= Vector-Informatik, according to HIS)
Module ID:	CRYPTO_MODULE_ID	114 decimal (according to ref. [3])

* For the detailed functional specification please also refer to the corresponding AUTOSAR SWS.

The Crypto Driver (CRYPTO) is called by the Crypto Interface (CRYIF) and performs the specific cryptographic functionality. The CRYPTO specification [1] offers a superset of algorithms which can be extended by ‘custom algorithms’. This software-based Crypto Driver offers a subset of algorithms and features which is described in 3.1.

2.1 Architecture Overview

The following figure shows where the CRYPTO is located in the AUTOSAR architecture.

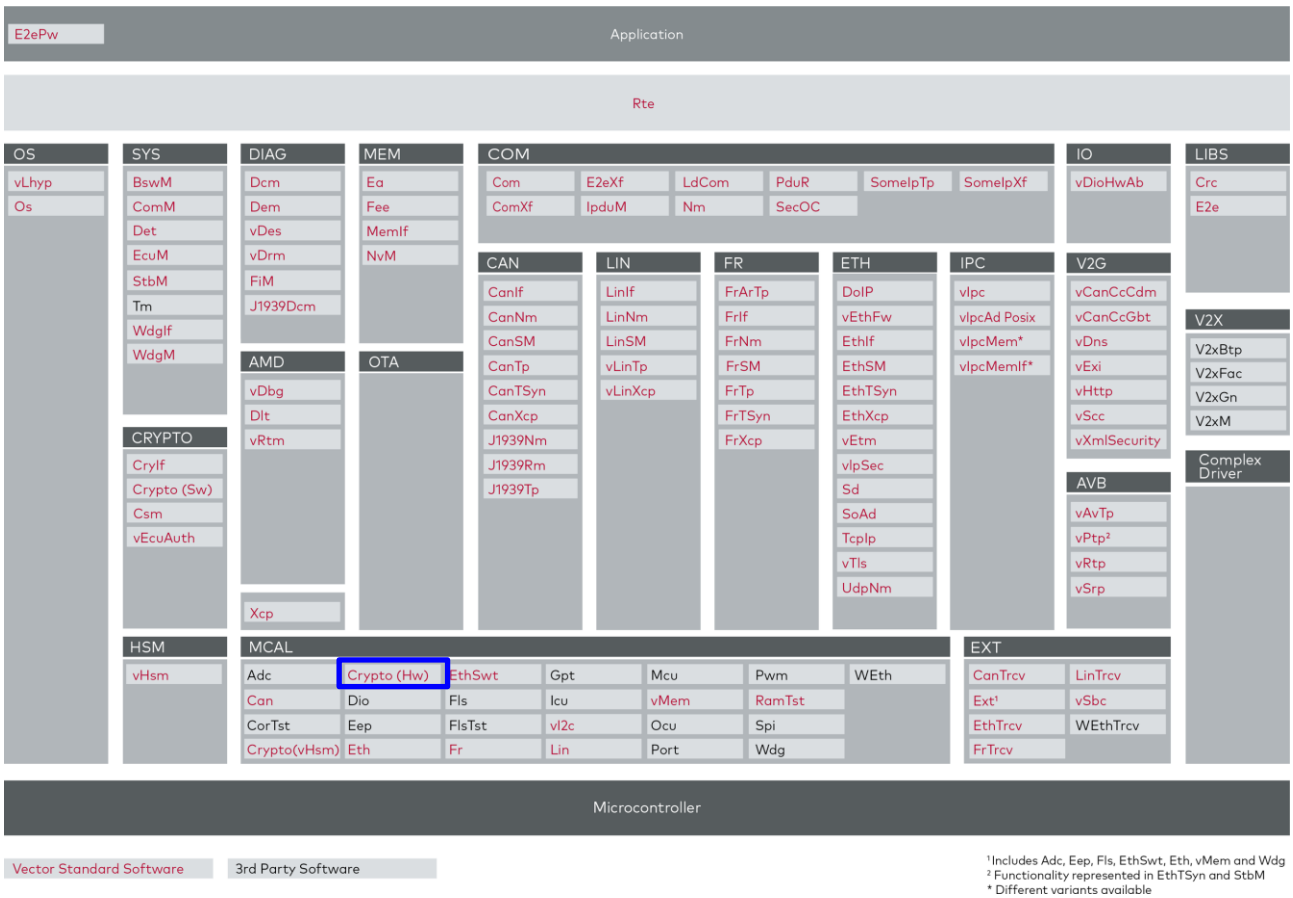


Figure 2-1 AUTOSAR 4.2 Architecture Overview

The next figure shows the interfaces to adjacent modules of the CRYPTO. These interfaces are described in chapter 5.

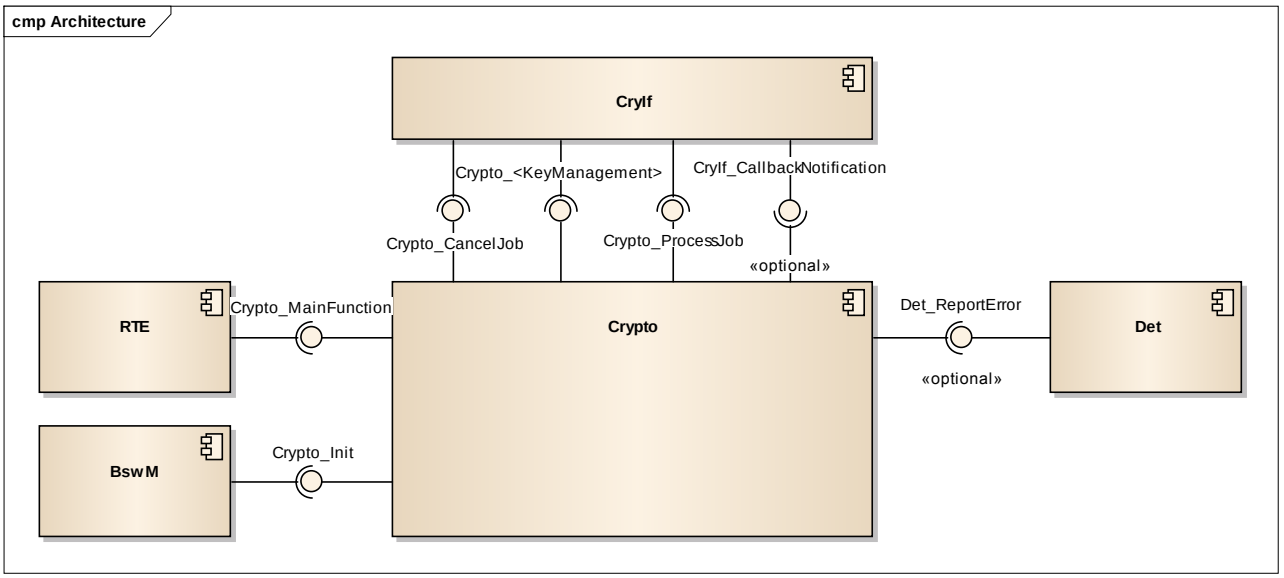


Figure 2-2 Interfaces to adjacent modules of the CRYPTO

3 Functional Description

3.1 Features

The features listed in the following tables cover the complete functionality specified for the CRYPTO.

The AUTOSAR standard functionality is specified in [1], the corresponding features are listed in the tables

> Table 3-1 Supported AUTOSAR standard conform features

> Table 3-2 Not supported AUTOSAR standard conform features

Vector Informatik provides further CRYPTO functionality beyond the AUTOSAR standard. The corresponding features are listed in the table

> Table 3-3 Features provided beyond the AUTOSAR standard

The following features specified in [1] are supported:

Supported AUTOSAR Standard Conform Features
Setting and Getting of Keys and its elements
Key Copying
Random Seed
Key Derive
Key Exchange
Symmetric Job Execution
Asymmetric Job Execution
Several symmetric and asymmetric Primitives

Table 3-1 Supported AUTOSAR standard conform features

3.1.1 Deviations

The following features specified in [1] are not supported:

Not Supported AUTOSAR Standard Conform Features
Certificate handling
Symmetric and asymmetric key generation
Cancelling of Jobs

Table 3-2 Not supported AUTOSAR standard conform features

3.1.2 Additions/ Extensions

The following features are provided beyond the AUTOSAR standard:

Features Provided Beyond The AUTOSAR Standard
Crypto_KeyElementSet supports an adapted variant of the ‘SHE Key Update Protocol’ [4]

Features Provided Beyond The AUTOSAR Standard
Retrieve version info from vHsm Firmware
Reading and setting reset vectors
Secure Boot
Flash operation synchronization
Secure Software Download

Table 3-3 Features provided beyond the AUTOSAR standard

3.1.3 Limitations

3.1.3.1 Cryptographic Algorithms and Modes

Only a subset of the stated algorithms and modes in the AUTOSAR SWS [1] are currently supported. The list of algorithms is described in Table 3-1.

3.1.3.2 Certificate Handling

Currently there is no implementation to parse and verify certificates. However, the API is still available for compatibility reasons.

3.1.3.3 Key Generation

Currently there is no implementation to generate keys. However, the API is still available for compatibility reasons.

3.1.3.4 AEAD Det Checks

The Det checks for AEAD differ from the table in [1]. There is a different handling required for the AEAD implementation. The parameter check is described in 3.5.2.

3.1.3.5 32 Bit Platform Support Only

Current implementation of the CRYPTO module is for 32 Bit platforms only. The usage with other platforms is prohibited.

3.2 Synchronization with vHsm

To synchronize the CRYPTO module with the vHsm, `Crypto_30_vHsm_WaitForHsmRam()` can be called in the startup code. This function will wait until the vHsm finished all initialization tasks and is ready to receive requests. The synchronization is accomplished using a special value written to the IPC.

Depending on the hardware platform, there might be other ways for synchronization. E.g. communication using a register.

3.3 Initialization

Before any other functionality of the CRYPTO module (except `Crypto_30_vHsm_WaitForHsmRam()`) can be called, the initialization function `Crypto_30_vHsm_Init()` has to be called by the BSWM.

For manual null initialization of RAM variables, the CRYPTO offers the function `Crypto_30_vHsm_InitMemory()` which can be called before `Crypto_30_vHsm_Init()`.

3.4 Main Functions

The CRYPTO module implementation provides one main function. When the usage of asynchronous job processing is enabled, this main function has to be called cyclically on task level. The main function is responsible to start processing of jobs.

3.5 Error Handling

3.5.1 Development Error Reporting (DET)

By default, development errors are reported to the DET using the service `Det_ReportError()` as specified in [2], if development error reporting is enabled (i.e. pre-compile parameter `CRYPTO_DEV_ERROR_REPORT == STD_ON`).

The driver can also forward DET errors occurred inside the vHsm if the vHsm is configured accordingly.

If another module is used for development error reporting, the function prototype for reporting the error can be configured by the integrator, but must have the same signature as the service `Det_ReportError()`.

The reported CRYPTO ID is 114.

The reported service IDs identify the services which are described in 5.1. The following table presents the service IDs and the related services:

Service ID	Service
0x00	Crypto_30_vHsm_Init()
0x01	Crypto_30_vHsm_GetVersionInfo()
0x03	Crypto_30_vHsm_ProcessJob()
0x0E	Crypto_30_vHsm_CancelJob()
0x04	Crypto_30_vHsm_KeyElementSet()
0x05	Crypto_30_vHsm_KeyValidSet()
0x06	Crypto_30_vHsm_KeyElementGet()
0x0F	Crypto_30_vHsm_KeyElementCopy()
0x10	Crypto_30_vHsm_KeyCopy()
0x11	Crypto_30_vHsm_KeyElementIdsGet()
0x0D	Crypto_30_vHsm_RandomSeed()
0x07	Crypto_30_vHsm_KeyGenerate()
0x08	Crypto_30_vHsm_KeyDerive()
0x09	Crypto_30_vHsm_KeyExchangeCalcPubVal()
0x0A	Crypto_30_vHsm_KeyExchangeCalcSecret()
0x0B	Crypto_30_vHsm_CertificateParse()
0x12	Crypto_30_vHsm_CertificateVerify()
0x0C	Crypto_30_vHsm_MainFunction()

Table 3-4 Service IDs

The errors reported to DET are described in the following table:

Error Code	Description
0x00	CRYPTO_E_UNINIT
0x01	CRYPTO_E_INIT_FAILED
0x02	CRYPTO_E_PARAM_POINTER
0x04	CRYPTO_E_PARAM_HANDLE
0x05	CRYPTO_E_PARAM_VALUE

Table 3-5 Errors reported to DET

3.5.2 Algorithm Parameter Overview

The required algorithm parameters are described in the following table. The required parameter differs from the table in [1].

Service \ Member	inputPtr	inputLength	secondaryInputPtr	secondaryInputLength	tertiaryInputPtr	tertiaryInputLength	outputPtr	outputLengthPtr	secondaryOutputPtr	secondaryOutputLengthPtr	verifyPtr	output64Ptr	mode
HASH	U	U					F	F					SUF
MACGENERATE	U	U					F	F					SUF
MACVERIFY	U	U	F	F							F		SUF
ENCRYPT	U	U					UF	UF					SUF
DECRYPT	U	U					UF	UF					SUF
AEADENCRYPT	U	U	V	U~			UF	UF	F	F			SUF
AEADDECRYPT	U	U	V	U~	F	F	UF	UF			F		SUF
SIGNATUREGENERATE	UF	U F~					F	F					SUF
SIGNATUREVERIFY	UF	U F~	F	F							F		SUF
SECCOUNTERINCREMENT													
SECCOUNTERREAD												F	
RANDOMGENERATE							F	F					

S: member required in Start mode.
U: member required in Update mode.
V: member optional in Update mode.
F: member required in Finish mode.
~: no Det check required / 0 is a valid value.

Table 3-6 Overview of the required algorithm parameter

3.6 Key Usage

To simplify the key usage this chapter describes some basic aspects.

3.6.1 Custom Key Elements

There are custom key elements defined, which are shown in the following table.

Custom Key Element	Value
CRYPTO_KE_CUSTOM_MAC_AES_ROUNDKEY	129
CRYPTO_KE_CUSTOM_KEYDERIVATION_LABEL	130
CRYPTO_KE_CUSTOM_RSA_MODULUS	160
CRYPTO_KE_CUSTOM_RSA_PUBLIC_EXPONENT	161
CRYPTO_KE_CUSTOM_RSA_PRIVATE_EXPONENT	162
CRYPTO_KE_CUSTOM_RSA_SALT	163
CRYPTO_KE_CUSTOM_RSA_SALT_LENGTH	164
CRYPTO_KE_CUSTOM_TLS_CLIENT_HELLO_RANDOM	3000
CRYPTO_KE_CUSTOM_TLS_SERVER_HELLO_RANDOM	3001
CRYPTO_KE_CUSTOM_TLS_HMAC_KEY_SIZE	3002
CRYPTO_KE_CUSTOM_KEYEXCHANGE_PARTNER_PUB_KEY	3003
CRYPTO_KE_CUSTOM_SCC_CONTRACT_PUBLIC_KEY	3013
CRYPTO_KE_CUSTOM_SCC_IV_AND_ENCRYPTED_PRIVATE_KEY	3014
CRYPTO_KE_CUSTOM_RANDOM_PERSONALIZATION_STRING	3015
CRYPTO_KE_CUSTOM_RANDOM_ADDITIONAL_INPUT	3016
CRYPTO_KE_CUSTOM_RANDOM_NONCE	3017
CRYPTO_KE_CUSTOM_RANDOM_RESEED_COUNTER	3018
CRYPTO_KE_CUSTOM_VHSM_VERSION	3020
CRYPTO_KE_CUSTOM_VHSM_UID	3021
CRYPTO_KE_CUSTOM_VHSM_PERFORM_PERSISTING	3022
CRYPTO_KE_CUSTOM_VHSM_ERRORLOG	3023
CRYPTO_KE_CUSTOM_VHSM_SECURE_BOOT_SLOT_ADDRESS	3024
CRYPTO_KE_CUSTOM_VHSM_SECURE_BOOT_SLOT_SIZE	3025
CRYPTO_KE_CUSTOM_VHSM_SECURE_BOOT_SLOT_CMAC	3026
CRYPTO_KE_CUSTOM_VHSM_SECURE_BOOT_SLOT_KEY	1
CRYPTO_KE_CUSTOM_VHSM_SECURE_BOOT_SLOT_SANCTION	3027
CRYPTO_KE_CUSTOM_VHSM_SECURE_BOOT_END	3028
CRYPTO_KE_CUSTOM_VHSM_BUNDLING_REQ	3029
CRYPTO_KE_CUSTOM_VHSM_BUNDLING_RESP	3030
CRYPTO_KE_CUSTOM_VHSM_BUNDLING_COUNTER	3031
CRYPTO_KE_CUSTOM_VHSM_FLASH_OPERATION	3032
CRYPTO_KE_CUSTOM_VHSM_RESET_VECTORS	3033
CRYPTO_KE_CUSTOM_VHSM_OPTION_BYTES	3034
CRYPTO_KE_CUSTOM_VHSM_UPGP	3035

CRYPTO_KE_CUSTOM_RSA_PRIME_P	3051
CRYPTO_KE_CUSTOM_RSA_PRIME_Q	3052
CRYPTO_KE_CUSTOM_RSA_EXPONENT_DP	3053
CRYPTO_KE_CUSTOM_RSA_EXPONENT_DQ	3054
CRYPTO_KE_CUSTOM_RSA_INVERSE_QI	3055

Table 3-7 Custom Key Elements

3.6.2 Key Element Values

Some key elements only accept specific values, which are shown in the following table.

Key Element	Define	Value
CRYPTO_KE_CUSTOM_VHSM_FLASH_OPERATION	CRYPTO_30_VHSM_DATAFLASH_START	0x00
	CRYPTO_30_VHSM_DATAFLASH_STOP	0x01
	CRYPTO_30_VHSM_CODEFLASH_START	0x02
	CRYPTO_30_VHSM_CODEFLASH_STOP	0x03
CRYPTO_KE_KEYEXCHANGE_ALGORITHM	CRYPTO_KEY_EXCHANGE_X25519	0x00
	CRYPTO_KEY_EXCHANGE_ANSIP256R1	0x01
CRYPTO_KE_KEYDERIVATION_ALGORITHM	CRYPTO_KDF_ALGO_KDF_SYM_NIST_800_108_CNT_MODE_SHA256	0x01
	CRYPTO_KDF_ALGO_KDF_ASYM_NIST_FIPS_186_4_ERB	0x02
	CRYPTO_KDF_ALGO_KDF_NIST_800_56_A_CONCATINATE_HASH	0x03
	CRYPTO_KDF_ALGO_KDF_ISO_15118	0x04

CRYPTO_KE_CUSTOM_VHSM_SECURE_BOOT_SLOT_SANCTION	CRYPTO_30_VHSM_SANCTION_NONE	0x01
	CRYPTO_30_VHSM_SANCTION_RESET	0x02
	CRYPTO_30_VHSM_SANCTION_CHANGE_RESET_VECTORS	0x03
	CRYPTO_30_VHSM_SANCTION_HALT	0x04

Table 3-8 Key Element Values

3.7 vHsm

This chapter describes specifics of the vHsm Firmware.

3.7.1 Pre Config File

The vHsm has the capability to generate a pre-config with the current settings of vHSM needed by CRYPTO, i.e. configured keys or driver objects including primitives.

This can be done for each IPC instance in the vHSM. The folder containing the pre-config file can be added to the list of additional definitions under the Project Settings of the application configuration.

3.7.2 Key Mapping

The key IDs on the Crypto driver and vHsm side must match each other.

To ensure this, use the generated pre-config file described in chapter 3.7.3 containing all accessible keys.



Note

Only the names of the keys and their IDs will be contained in the pre-config file.

The information about the CryptoKeyType and CryptoKeyElements are not used on the driver side and are therefore only dummies.

The actual configuration and their structure are configured inside the Crypto_30_LibCv driver inside the vHsm.

3.7.3 Crypto Driver Objects

The Crypto Driver Objects, especially the ID, of CRYPTO and vHsm side must match each other.

To ensure this, use the generated pre-config file described in chapter 3.7.3 containing the driver objects including their provided primitives.

3.7.4 IPC Versions

The CRYPTO and the vHSM communicate over a RAM based IPC protocol. However, this protocol has a version which needs to be the same on both sides.

For HSM versions starting from Protocol Version 2, the version is synchronized via the generated pre-config file described in chapter 3.7.3.

For earlier versions, set this parameter manually to 1.

Protocol Version 1 only allows the usage of 4 driver objects of the vHsm:

- **Core:** Can be used to issue the secure boot and secure software update jobs
- **LibCv:** Can be used to compute algorithms inside the software implementation on the vHsm. The available primitives depend on the vHsm configuration and can be synchronized with a pre-config file generated by the vHsm.
- **Hal:** Can be used to compute algorithms in hardware such as CMAC, AES and TRNG
- **Internal:** Cannot be used to perform jobs. This driver object is internally used to send the key management API calls to the vHsm. Do not reference this driver object in the CryIf.

With Protocol Version 2 and higher, there is no limit for driver objects anymore.

3.7.5 Memory Access

Input and output data for the vHsm must be located either in the code flash or global RAM of the derivative. The vHsm cannot access the local RAM of the application cores. If the CRYPTO driver detects that the provided input or output memory location is not usable, it will copy the data to a buffer in the global RAM instead.



Basic Knowledge
The buffer of the Crypto_30_vHSM needs to be located in global RAM and needs to be ensured in the Memory Map!



Expert Knowledge
A big buffer on the global RAM and unnecessary copy operations can be avoided by ensuring that the data is in a valid input memory region.

3.7.6 Special services

3.7.6.1 Version Info

Call Crypto_30_vHsm_KeyElementGet() to retrieve the key element `CRYPTO_KE_CUSTOM_VHSM_VERSION` from the info key. The version consists of three bytes which are the concatenation of `MAJOR|MINOR|BUGFIX`.

3.7.6.2 UID

Call the function `Crypto_30_vHsm_KeyElementGet()` to retrieve the key element `CRYPTO_KE_CUSTOM_VHSM_UID` from the info key. The info key is the key which is referenced under `/vHsm/vHsmKeyMapping/vHsmInfoKey` in the vHsm configuration.

3.7.6.3 Error Log

Call the function `Crypto_30_vHsm_KeyElementGet()` to retrieve the key element `CRYPTO_KE_CUSTOM_VHSM_ERRORLOG` from the info key.

The latest error will be written to the beginning. An error entry consists of two uint32 values (errorId followed by the errorData). Older errors will be moved back until there is no more space in the key available, which will discard the oldest log entry.

3.7.6.4 Secure Boot

Set the following key elements of a secure boot slot:

- CRYPTO_KE_CUSTOM_VHSM_SECURE_BOOT_SLOT_ADDRESS
- CRYPTO_KE_CUSTOM_VHSM_SECURE_BOOT_SLOT_SIZE
- CRYPTO_KE_CUSTOM_VHSM_SECURE_BOOT_SLOT_KEY
- Optionally CRYPTO_KE_CUSTOM_VHSM_SECURE_BOOT_SLOT_SANCTION

Call a `Csm_MacVerify()` with a job referencing a primitive which has the algorithm family set to `CRYPTO_ALGOFAM_CUSTOM_SECURE_BOOT`.

The variable `macPtr` is ignored and can be a dummy variable.

The variable pointed by `verifyPtr` will be filled with the result of the secure boot check in case `VHSM_SECURE_BOOT_ALLOW_JOB_PROCESSING` is set to `STD_OFF`.

The CMAC of the secure boot slot will be filled during the first call. All subsequent calls will be checked against that generated CMAC.

Depending on the reset vector, the first or second secure boot slot will be checked automatically at the startup of the vHsm.

The sanction on a failed secure boot is optional and is stored in the key element with the id CRYPTO_KE_CUSTOM_VHSM_SECURE_BOOT_SLOT_SANCTION.



Caution
Secure Boot cannot be done from address 0x00000000.
The next possible address is 0x00000200.
Set the reset vectors and the first secure boot slot accordingly.

This is because AUTOSAR specifies NULL_PTR checks inside the whole stack which prevents using 0x00000000 as legitimate address.

Key Element	Size	Persist	Read Access	Write Access	Info
SLOT_ADDRESS	4	TRUE	Allow	Denied	Init Value: e.g. `0x00 0x02 0x00 0x00` for address `0x00000200`
SLOT_SIZE	4	TRUE	Allow	Denied	Init Value: e.g. `0x10 0x00 0x00 0x00` for a size of 16 bytes
SLOT_KEY	16	TRUE	Denied	Allowed	
SLOT_SANCTION	1	FALSE	Denied	Denied	Init Value: Set the init value to the needed sanction.
SLOT_CMAC	16	TRUE	Denied	Denied	Partial Access: TRUE Init Value: 1 byte: `0x00`

Table 3-9 Recommended settings for secure boot key elements

3.7.6.5 Secure Software Download

To perform a secure software download, which means updating an already stored CMAC in a secure boot slot, call `Csm_SignatureVerify()` with a job, which has the algorithm family set to `CRYPTO_ALGOFAM_CUSTOM_SOFTWARE_DOWNLOAD`.

The provided signature will be validated against the key which is referenced in `vHsm/vHsmKeyMapping/vHsmSecureSoftwareDownloadKey` inside the vHsm Configuration.

If the signature over the provided data is valid, the CMAC, address and size of the secure boot slot will be updated.

The variable pointed by verifyPtr will be filled with the result of the signature verification.



Note
This is not a basic feature, is hardware dependent and is not included in every vHSM firmware.

3.7.6.6 Hardware Dependent Features

There might be further features, e.g. setting reset vectors, flash synchronization or changing the upgrade protection. These features are strongly hardware dependent. Please refer to the corresponding technical reference of vHSM to get to know which features are available and how to use it.

3.7.7 Limitations

3.7.7.1 Secure Boot

Secure Boot cannot be done from Address zero (NULL_PTR). This is because the AUTOSAR APIs specify checks against NULL_PTR at various locations inside the stack.

Set the application core reset vectors to e.g. 0x00000200.

3.7.7.2 Write-Once Keys

Only KeyElementSet and KeyElementCopy can be used with write-once keys.

3.8 Wait Loop Callout

The CRYPTO has the configurable option to enable a callout which is called as long as the CRYPTO is waiting for the result of an synchronous job or key function. The callout is described in chapter 5.3.1.



Caution
The callout function needs to be mapped to RAM in case the application uses vHSM features which require the HSM to access the code flash.

4 Integration

This chapter gives necessary information for the integration of the MICROSAR CRYPTO into an application environment of an ECU.

4.1 Scope of Delivery

The delivery of CRYPTO contains the files which are described in chapter 4.1.1 and 4.1.2:

4.1.1 Static Files

File Name	Description
Crypto_30_vHsm.c	This is the main source file of the CRYPTO
Crypto_30_vHsm.h	This is the source file of the CRYPTO
Crypto_30_vHsm_Custom.h	This header contains driver specific custom defines for key elements and algorithm families or modes.
Crypto_30_vHsm_Ipc.c	This source file is used for the IPC to the vHsm
Crypto_30_vHsm_Ipc.h	This header file is used for the IPC to the vHsm
Crypto_30_vHsm_Ipc_Types.h	This header file is used for the IPC to the vHsm
Crypto_30_vHsm_Irq.c	This source file contains the interrupt service routine
Crypto_30_vHsm_Jobs.c	This source file is used for providing jobs to the vHsm
Crypto_30_vHsm_Jobs.h	This header file is used for providing jobs to the vHsm
Crypto_30_vHsm_KeyManagement.c	This source contains the CRYPTO´s key management functions
Crypto_30_vHsm_KeyManagement.h	This header contains the CRYPTO´s key management functions
Crypto_30_vHsm_TransformerLib.h	This header file is used for the read and write access to the IPC.
Crypto_30_vHsm_Types.h	This header contains types.

Table 4-1 Static files

4.1.2 Dynamic Files

The dynamic files are generated by the configuration tool DaVinci Configurator 5 Pro.

File Name	Description
Crypto_30_vHsm_Cfg.c	This is the configuration source file.
Crypto_30_vHsm_Cfg.h	This is the configuration header file.

Table 4-2 Generated files

4.2 Critical Sections

Crypto uses the following critical sections:

> CRYPTO_30_VHSM_EXCLUSIVE_AREA_0

This critical section protects global data from inconsistencies

> CRYPTO_30_VHSM_EXCLUSIVE_AREA_1

This critical section ensures that a forwarded DET from the vHsm is processed only once (Flag is located in the IPC memory)

See also comments of internal behavior in DaVinci Configurator for further information.

5 API Description

For an interfaces overview please see Figure 2-2.

5.1 Services provided by CRYPTO

5.1.1 Crypto_30_vHsm_Init

Prototype	
`void Crypto_30_vHsm_Init (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
Initializes the Crypto Driver.	
Particularities and Limitations	
Specification of module initialization > Interrupts are disabled. Module is uninitialized. This function initializes the module Crypto_30_vHsm. It initializes all variables and sets the module state to initialized.	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 5-1 Crypto_30_vHsm_Init

5.1.2 Crypto_30_vHsm_InitMemory

Prototype	
`void Crypto_30_vHsm_InitMemory (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
The function initializes variables, which cannot be initialized with the startup code.	
Particularities and Limitations	
Module is uninitialized. Initialize component variables at power up.	

Call context
> TASK > This function is Synchronous > This function is Non-Reentrant

Table 5-2 Crypto_30_vHsm_InitMemory

5.1.3 Crypto_30_vHsm_GetVersionInfo

Prototype	
`void Crypto_30_vHsm_GetVersionInfo (Std_VersionInfoType *versioninfo)`	
Parameter	
versioninfo [out]	Pointer to where to store the version information. Parameter must not be NULL.
Return code	
void	none
Functional Description	
Returns the version information.	
Particularities and Limitations	
none Crypto_30_vHsm_GetVersionInfo() returns version information, vendor ID and AUTOSAR module ID of the component.	
Call context	
> TASK|ISR > This function is Synchronous > This function is Reentrant	

Table 5-3 Crypto_30_vHsm_GetVersionInfo

5.1.4 Crypto_30_vHsm_ProcessJob

Prototype	
`Std_ReturnType Crypto_30_vHsm_ProcessJob (uint32 objectId, Crypto_JobType *job)`	
Parameter	
objectId [in]	Holds the identifier of the Crypto Driver Object.
job [in,out]	Pointer to the configuration of the job. Contains structures with job and primitive relevant information but also pointer to result buffers.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.

	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
	CRYPTO_E_KEY_NOT_VALID Request failed, the key is not valid.
	CRYPTO_E_QUEUE_FULL Request failed, the queue is full.
	CRYPTO_E_SMALL_BUFFER Request failed, the provided buffer is too small to store the result.
Functional Description	
Process the received job.	
Particularities and Limitations	
none Performs the crypto primitive that is configured in the job parameter.	
Call context	
> TASK > This function is Reentrant	

Table 5-4 Crypto_30_vHsm_ProcessJob

5.1.5 Crypto_30_vHsm_CancelJob

Prototype	
`Std_ReturnType` **`Crypto_30_vHsm_CancelJob`** `(uint32 objectId, Crypto_JobType *job)`	
Parameter	
objectId [in]	Holds the identifier of the Crypto Driver Object.
job [in,out]	Pointer to the configuration of the job. Contains structures with user and primitive relevant information.
Return code	
Std_ReturnType	E_OK Request successful, job has been removed.
Std_ReturnType	E_NOT_OK Request failed, job could not be removed.
Functional Description	
Cancels the received job.	
Particularities and Limitations	
none This interface removes the provided job from the queue and cancels the processing of the job if possible.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-5 Crypto_30_vHsm_CancelJob

5.1.6 Crypto_30_WaitForHsmRam

Prototype	
`Void` **`Crypto_30_vHsm_WaitForHsmRam`** `(void)`	
Parameter	
None	
Return code	
None	
Functional Description	
Waits until vHsm is ready after reset.	
Particularities and Limitations	
This function is mapped to RAM to avoid conflicting access to code flash with the vHsm. Can be called before Crypto_30_vHsm_Init(). Can be called from Startup Code, None.	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 5-6 Crypto_30_vHsm_WaitForHsmRam

5.2 Key Management Functions

5.2.1 Crypto_30_vHsm_KeyCopy

Prototype	
`Std_ReturnType` **`Crypto_30_vHsm_KeyCopy`** `(uint32 cryptoKeyId, uint32 targetCryptoKeyId)`	
Parameter	
cryptoKeyId [in]	Holds the identifier of the key whose key element shall be the source element.
targetCryptoKeyId [in]	Holds the identifier of the key whose key element shall be the destination element.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
	CRYPTO_E_KEY_READ_FAIL Request failed, read access was denied.
	CRYPTO_E_KEY_WRITE_FAIL Request failed, write access was denied.

	CRYPTO_E_KEY_NOT_AVAILABLE Request failed, the key is not available.
	CRYPTO_E_KEY_SIZE_MISMATCH Request failed, the key element sizes are not compatible.
Functional Description	
Copy the key.	
Particularities and Limitations	
Copies a key with all its elements to another key in the same crypto driver. The key element can only be copied, if the destination key element write access right is less than WA_INTERNAL_COPY. Additional, the read access right of the source must be less than RA_INTERNAL_COPY and the destination read access right must be higher or equal than the source read access right.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant for different crypto keys.	

Table 5-7 Crypto_30_vHsm_KeyCopy

5.2.2 Crypto_30_vHsm_KeyElementCopy

Prototype	
`Std_ReturnType` **`Crypto_30_vHsm_KeyElementCopy`** `(uint32 cryptoKeyId, uint32 keyElementId, uint32 targetCryptoKeyId, uint32 targetKeyElementId)`	
Parameter	
cryptoKeyId [in]	Holds the identifier of the key whose key element shall be the source element.
keyElementId [in]	Holds the identifier of the key element which shall be the source for the copy operation.
targetCryptoKeyId [in]	Holds the identifier of the key whose key element shall be the destination element.
targetKeyElementId [in]	Holds the identifier of the key element which shall be the destination for the copy operation.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
	CRYPTO_E_KEY_READ_FAIL Request failed, read access was denied.
	CRYPTO_E_KEY_WRITE_FAIL Request failed, write access was denied.
	CRYPTO_E_KEY_EXTRACT_DENIED Request failed, not allowed to extract key material.
	CRYPTO_E_KEY_NOT_AVAILABLE Request failed, the key is not available.
	CRYPTO_E_KEY_SIZE_MISMATCH Request failed, the key element sizes are not compatible.

Functional Description
Copy key element.
Particularities and Limitations
Copies a key element to another key element in the same crypto driver. The key element can only be copied, if the destination key element write access right is less than WA_INTERNAL_COPY. Additional, the read access right of the source must be less than RA_INTERNAL_COPY and the destination read access right must be higher or equal than the source read access right.
Call context
> TASK > This function is Synchronous > This function is Reentrant for different crypto keys.

Table 5-8 Crypto_30_vHsm_KeyElementCopy

5.2.3 Crypto_30_vHsm_KeyElementIdsGet

Prototype	
`Std_ReturnType` **`Crypto_30_vHsm_KeyElementIdsGet`** `(uint32 cryptoKeyId, uint32 *keyElementIdsPtr, uint32 *keyElementIdsLengthPtr)`	
Parameter	
cryptoKeyId [in]	Holds the identifier of the key whose available element ids shall be exported.
keyElementIdsLengthPtr [in]	Holds a pointer to the memory location in which the number of key element in the given key is stored. On calling this function, this parameter shall contain the size of the buffer provided by keyElementIdsPtr. When the request has finished, the actual number of key elements is stored.
keyElementIdsPtr [out]	Contains the pointer to the array where the ids of the key elements shall be stored.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
	CRYPTO_E_SMALL_BUFFER Request failed, the provided buffer is too small to store the result.
Functional Description	
Used to retrieve information which key elements are available in a given key.	
Particularities and Limitations	
none -	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-9 Crypto_30_vHsm_KeyElementIdsGet

5.2.4 Crypto_30_vHsm_KeyElementSet

Prototype	
`Std_ReturnType` **`Crypto_30_vHsm_KeyElementSet`** `(uint32 cryptoKeyId, uint32 keyElementId, const uint8 *keyPtr, uint32 keyLength)`	
Parameter	
cryptoKeyId [in]	Holds the identifier of the key whose key element shall be set.
keyElementId [in]	Holds the identifier of the key element which shall be set.
keyPtr [in]	Holds the pointer to the key data which shall be set as key element.
keyLength [in]	Contains the length of the key element in bytes.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
	CRYPTO_E_KEY_WRITE_FAIL Request failed, write access was denied.
	CRYPTO_E_KEY_NOT_AVAILABLE Request failed, the key is not available.
	CRYPTO_E_KEY_SIZE_MISMATCH Request failed, the key element size does not match size of provided data.
Functional Description	
Sets a key element.	
Particularities and Limitations	
Sets the given key element bytes to the key identified by cryptoKeyId. The key element can only be set, if the write access right is WA_ALLOWED or WA_ENCRYPTED for she key update.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant for different crypto keys.	

Table 5-10 Crypto_30_vHsm_KeyElementSet

5.2.5 Crypto_30_vHsm_KeyValidSet

Prototype	
`Std_ReturnType` **`Crypto_30_vHsm_KeyValidSet`** `(uint32 cryptoKeyId)`	
Parameter	
cryptoKeyId [in]	Holds the identifier of the key whose key elements shall be set to valid.
Return code	
Std_ReturnType	E_OK Request successful.

	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
Functional Description	
Sets the key to valid.	
Particularities and Limitations	
none Sets the key state of the key identified by cryptoKeyId to valid.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant for different crypto keys.	

Table 5-11 Crypto_30_vHsm_KeyValidSet

5.2.6 Crypto_30_vHsm_KeyElementGet

Prototype	
`Std_ReturnType` **`Crypto_30_vHsm_KeyElementGet`** `(uint32 cryptoKeyId, uint32 keyElementId, uint8 *resultPtr, uint32 *resultLengthPtr)`	
Parameter	
cryptoKeyId [in]	Holds the identifier of the key whose key element shall be set.
keyElementId [in]	Holds the identifier of the key element which shall be set.
resultPtr [in]	Holds the pointer to the key data which shall be set as key element.
resultLengthPtr [in]	Contains the length of the key element in bytes.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
	CRYPTO_E_KEY_READ_FAIL Request failed, read access was denied.
	CRYPTO_E_KEY_NOT_AVAILABLE Request failed, the key is not available.
	CRYPTO_E_KEY_SIZE_MISMATCH Request failed, the provided buffer is too small to store the result.
Functional Description	
This interface shall be used to get a key element of the key identified by the cryptoKeyId and store the key element in the memory location pointed by the result pointer.	
Particularities and Limitations	
The key element can only be provided, if the read access right is RA_ALLOWED.	
Call context	
> TASK > This function is Synchronous	

> This function is Reentrant

Table 5-12 Crypto_30_vHsm_KeyElementGet

5.2.7 Crypto_30_vHsm_RandomSeed

Prototype	
`Std_ReturnType` **`Crypto_30_vHsm_RandomSeed`** `(uint32 cryptoKeyId, const uint8 *entropyPtr, uint32 entropyLength)`	
Parameter	
cryptoKeyId [in]	Holds the identifier of the key for which a new seed shall be generated.
entropyPtr [in]	Holds a pointer to the memory location which contains the data to feed the entropy.
entropyLength [in]	Contains the length of the entropy in bytes.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
	CRYPTO_E_SMALL_BUFFER Request failed, the provided buffer is too small to store the result.
Functional Description	
Initialize the seed.	
Particularities and Limitations	
none This function generates the internal seed state using the provided entropy source. Furthermore, this function can be used to update the seed state with new entropy.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant for different crypto keys.	

Table 5-13 Crypto_30_vHsm_RandomSeed

5.2.8 Crypto_30_vHsm_KeyGenerate

Prototype	
`Std_ReturnType` **`Crypto_30_vHsm_KeyGenerate`** `(uint32 cryptoKeyId)`	
Parameter	
cryptoKeyId [in]	Holds the identifier of the key which is to be updated with the generated value.

Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
Functional Description	
Generates a key.	
Particularities and Limitations	
none This function shall dispatch the key generate function to the configured crypto driver object.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant for different crypto keys.	

Table 5-14 Crypto_30_vHsm_KeyGenerate

5.2.9 Crypto_30_vHsm_KeyDerive

Prototype	
`Std_ReturnType` **`Crypto_30_vHsm_KeyDerive`** `(uint32 cryptoKeyId, uint32 targetCryptoKeyId)`	
Parameter	
cryptoKeyId [in]	Holds the identifier of the key which is used for key derivation.
targetCryptoKeyId [in]	Holds the identifier of the key which is used to store the derived key.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
Functional Description	
Derives a key.	
Particularities and Limitations	
Derives a new key by using the key elements in the given key identified by the cryptoKeyId. The given key must contain the key elements for the password (CRYPTO_KE_KEYDERIVATION_PASSWORD), salt (CRYPTO_KE_KEYDERIVATION_SALT), algorithm (CRYPTO_KE_KEYDERIVATION_ALGORITHM) and the custom element label(CRYPTO_KE_CUSTOM_KEYDERIVATION_LABEL). The number of iterations is automatically determined by the needed key length. The derived key is stored in the key element with id 1 of the given key identified by targetCryptoKeyId. The write access right for the destination key element must be less or equal than WA_INTERNAL_COPY.	
Call context	
> TASK > This function is Synchronous	

> This function is Reentrant for different crypto keys.

Table 5-15 Crypto_30_vHsm_KeyDerive

5.2.10 Crypto_30_vHsm_KeyExchangeCalcPubVal

Prototype	
`Std_ReturnType` **`Crypto_30_vHsm_KeyExchangeCalcPubVal`** `(uint32 cryptoKeyId, uint8 *publicValuePtr, uint32 *publicValueLengthPtr)`	
Parameter	
cryptoKeyId [in]	Holds the identifier of the key which shall be used for the key exchange protocol.
publicValuePtr [out]	Contains the pointer to the data where the public value shall be stored.
publicValueLengthPtr [in,out]	Holds a pointer to the memory location in which the public value length information is stored. On calling this function, this parameter shall contain the size of the buffer provided by publicValuePtr. When the request has finished, the actual length of the returned value shall be stored.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
	CRYPTO_E_SMALL_BUFFER Request failed, the provided buffer is too small to store the result.
Functional Description	
Calculation of the public value.	
Particularities and Limitations	
Calculates the public value for the key exchange and stores the public key in the memory location pointed by the public value pointer. The write access right for the destination key element must be less or equal than WA_INTERNAL_COPY.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant for different crypto keys.	

Table 5-16 Crypto_30_vHsm_KeyExchangeCalcPubVal

5.2.11 Crypto_30_vHsm_KeyExchangeCalcSecret

Prototype
`Std_ReturnType` **`Crypto_30_vHsm_KeyExchangeCalcSecret`** `(uint32 cryptoKeyId, const uint8 *partnerPublicValuePtr, uint32 partnerPublicValueLength)`

Parameter	
cryptoKeyId [in]	Holds the identifier of the key which shall be used for the key exchange protocol.
partnerPublicValuePtr [in]	Holds the pointer to the memory location which contains the partners public value.
partnerPublicValueLength [in]	Contains the length of the partners public value in bytes.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
	CRYPTO_E_SMALL_BUFFER Request failed, the provided buffer is too small to store the result.
Functional Description	
Calculation of the secret.	
Particularities and Limitations	
Calculates the shared secret key for the key exchange with the key material of the key identified by the cryptoKeyId and the partner public key. The shared secret key is stored as a key element in the same key. The write access right for the destination key element must be less or equal than WA_INTERNAL_COPY.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant for different crypto keys.	

Table 5-17 Crypto_30_vHsm_KeyExchangeCalcSecret

5.2.12 Crypto_30_vHsm_CertificateParse

Prototype	
`Std_ReturnType` **`Crypto_30_vHsm_CertificateParse`** `(uint32 cryptoKeyId)`	
Parameter	
cryptoKeyId [in]	Holds the identifier of the key slot in which the certificate has been stored.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
Functional Description	
Parse stored certificate.	
Particularities and Limitations	
none	

Parses the certificate data stored in the key element CRYPTO_KE_CERT_DATA and fills the key elements CRYPTO_KE_CERT_SIGNEDDATA, CRYPTO_KE_CERT_PARSEDPUBLICKEY and CRYPTO_KE_CERT_SIGNATURE
Call context
> TASK > This function is Synchronous > This function is Reentrant for different crypto keys.

Table 5-18 Crypto_30_vHsm_CertificateParse

5.2.13 Crypto_30_vHsm_CertificateVerify

Prototype	
`Std_ReturnType` **`Crypto_30_vHsm_CertificateVerify`** `(uint32 cryptoKeyId, uint32 verifyCryptoKeyId, Crypto_VerifyResultType *verifyPtr)`	
Parameter	
cryptoKeyId [in]	Holds the identifier of the key which shall be used to validate the certificate.
verifyCryptoKeyId [in]	Holds the identifier of the key containing the certificate, which shall be verified.
verifyPtr [out]	Holds a pointer to the memory location which will contain the result of the certificate verification.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
Functional Description	
Certificate verification.	
Particularities and Limitations	
none Verifies the certificate stored in the key referenced by verifyCryptoKeyId with the certificate stored in the key referenced by cryptoKeyId.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant for different crypto keys.	

Table 5-19 Crypto_30_vHsm_CertificateVerify

5.3 Callout Functions

CRYPTO defines some configurable callout functions. The declarations of the callout functions are provided by the CRYPTO module. It is the integrator's task to provide the corresponding function definitions. The definitions of the callouts can be adjusted to the

system's needs. The CRYPTO callout function declarations are described in the following tables:

5.3.1 Wait Loop Callout

Prototype	
`void` **`<WaitLoopCalloutName>`**`(void)`	
Parameter	
`void`	-
Return code	
`void`	none
Functional Description	
Notification about ongoing waiting for a response from the vHsm Firmware for a synchronous job.	
Particularities and Limitations	
> Implementation has to be done in the application. > The name of this function is configurable. > If HSM is required to execute code flash operations, this functions needs to be mapped to RAM.	
Call context	
> Interrupt or task context > Callout is invoked whenever driver is waiting for a synchronous job > This function is Synchronous > This function is Non-Reentrant	

Table 5-20 Key Verification Callout

5.4 Services needed by CRYPTO

No services are needed for CRYPTO.

5.5 Services used by CRYPTO

In the following table services provided by other components, which are used by the CRYPTO are listed. For details about prototype and functionality refer to the documentation of the providing component.

Component	API
DET	Det_ReportError
CRYIF	CryIf_CallbackNotification

Table 5-21 Services used by the CRYPTO

6 Configuration

In the CRYPTO the attributes can be configured according to/ with the following methods/ tools:

- Configuration in DaVinci Configurator 5 Pro

6.1 Configuration Variants

The CRYPTO supports the configuration variants

- VARIANT-PRE-COMPILE

The configuration classes of the CRYPTO parameters depend on the supported configuration variants. For their definitions please see the Crypto_30_vHsm_bswmd.arxml file.

7 Glossary and Abbreviations

7.1 Glossary

Term	Description
CSM	Crypto Service Manager
CRYIF	Crypto Interface
CRYPTO	Crypto Driver

Table 7-1 Glossary

7.2 Abbreviations

Abbreviation	Description
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSW	Basis Software
DEM	Diagnostic Event Manager
DET	Development Error Tracer
EAD	Embedded Architecture Designer
ECU	Electronic Control Unit
HIS	Hersteller Initiative Software
ISR	Interrupt Service Routine
MICROSAR	Microcontroller Open System Architecture (the Vector AUTOSAR solution)
PPORT	Provide Port
RPORT	Require Port
RTE	Runtime Environment
SHE	Secure Hardware Extension
SRS	Software Requirement Specification
SWC	Software Component
SWS	Software Specification

Table 7-2 Abbreviations

8 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com